goapi rutracker keystone — deep multi-layer diagnosis (2026-07-02)

Device-proven root-cause chain for why the `run-matrix --backend goapi --subsets rutracker --queries 1080p --external-backend` keystone still FAILS at search, even after the CASE-COOKIE session-token fixes (A+B) landed. All evidence captured on a real containerized-KVM emulator (API 34, x86_64) against a real standalone lava-api-go on `https://127.0.0.1:8443` reached via `adb reverse tcp:8443 tcp:8443`.

What WORKS (proven)

- Onboarding completes; the ApiSelection probe to `127.0.0.1:8443` succeeds (advances to Providers) — so the adb-reverse tunnel + Go API `/health` are fine.
- rutracker LOGIN succeeds via the on-device DIRECT bundled client: `RuTrackerHttp: REQUEST login.php → profile.php?…u=47500467`, `OnboardingViewModel: cred path: login(rutracker)=Authenticated, bb_session=…`.
- The Go API is server-side correct: `GET /providers` (no auth) returns the catalogue with rutracker declaring SEARCH; a keyed `/v1/rutracker/search` returns 200 + 50 results (proven by the earlier isolated root-cause).

What FAILS, and exactly why (instrumentation-proven)

Temporary `System.err.println("GOAPI-DEBUG …")` probes at the search boundary proved the routing (all reverted after diagnosis):

- `SRVM.onCreate query=1080p providerIds=[rutracker]` → `observeStreamMultiSearch` is correctly entered (the `pids` nav-arg parses fine; NOT the legacy paging path).
- `multiSearch id=rutracker clientClass=RuTrackerClient featureNull=false` + `caps=[SEARCH,BROWSE,FORUM,TOPIC,COMMENTS,FAVORITES,TORRENT_DOWNLOAD,MAGNET_LINK,AUTH_REQUIRED,CAPTCHA_LOGIN]` → `clientFor("rutracker")` resolves to the **BUNDLED**

RuTrackerClient, NOT the dynamic Go-API-backed ApiBackedTrackerClient. (The caps are the bundled descriptor's — note FORUM not the catalogue's FORUM_TREE, and no RSS/UPLOAD/USER_PROFILE.)
- No GOAPI-DEBUG search … marker fired → ApiBackedTrackerClient.search() was never called. The bundled RuTrackerClient.search() ran and made NO HTTP request to rutracker.org (only the login requests appear) → error.

The layer chain

1. **LAYER 0 (fixed, A+B):** even when the ApiBackedTrackerClient IS used, the onboarding-obtained bb_session did not propagate to its Auth-Token. Fixed + reproduce-first tested. See docs/issues/fixed/BUGFIXES.md (2026-07-02 CASE-COOKIE).
2. **LAYER 1 (open):** the external Go API endpoint is onboarded KEYLESS — withLocalApiKeyIfMissing() only supplies a key for the on-device api-app, not a cloud/external standalone lava-api-go. So fetchAndPopulateProviders → GET /v1/providers returns **401** → populateFrom keeps the BUNDLED providers (documented fallback, OnboardingViewModel.kt:337-339) → rutracker resolves to the bundled direct client, never routing through the Go API. UNCONFIRMED: whether the intended remedy is client-side key provisioning, a harness-injected key, or an open/test-auth Go API mode — tracked by the follow-up investigation.
3. **LAYER 2 (open):** the bundled RuTrackerClient.search() fallback itself fails after a successful bundled login, making NO tracker.php request — a guard short-circuits before the request. UNCONFIRMED: exact guard (follow-up).

Recommended next step

Provision the Lava-Auth key for the external Go API so the dynamic client registers and search routes through the Go API (then A+B apply) — OR fix the bundled fallback search. Deep-investigation of both layers is in flight; findings land in /run/media/milosvasic/DATA4TB/.build-tmp/goapi-keystone-layers-2026-07-02.md.

Anti-bluff posture

The keystone is NOT green and is NOT claimed green. A+B are committed as verified fixes for the ApiBackedTrackerClient session-token path (real, reproduce-first, Bluff-Audited), with this

document recording the still-open layers and their physical evidence. Per §6.J/§6.AK: no distribute until the matrix is genuinely green.